Aula 06 — Tipos de Dados

Respostas do exercício em duplas

Disciplina: Paradigmas de Linguagens de Programação
Base: Aula 06 — Tipos de Dados

Questão 1 — JavaScript

Código:

```
console.log(0.1 * 3 === 0.3);
console.log(9007199254740993);
```

Resposta:
A primeira linha imprime false. O cálculo 0.1 * 3 resulta em 0.30000000000000004, portanto não é exatamente igual a 0.3.
A segunda linha imprime 9007199254740992. Isso ocorre porque o Number do JavaScript usa ponto flutuante de 64 bits e perde precisão para inteiros acima de 2^53.
Detecção: o problema nunca é detectado automaticamente.

Questão 2 — Python

Código:

```
p = "maçã"
print(len(p), len(p.encode()))
```

Resposta:
A saída é 4 6. A palavra possui 4 caracteres, mas ocupa 6 bytes em UTF-8, pois “ç” e “ã” ocupam 2 bytes cada.
Detecção: não há erro; é uma diferença entre contar caracteres e contar bytes.

Questão 3 — Go

Código:

```
var b byte = 255
b++
fmt.Println(b)
```

Resposta:

A saída é 0. Em Go, byte é um uint8, cuja faixa vai de 0 a 255. Ao incrementar 255, ocorre estouro e o valor dá a volta para 0.
Detecção: o estouro é silencioso e não é detectado automaticamente.

Questão 4 — Java

Código:

```
int[] v = new int[3];
System.out.println(v[0]);
System.out.println(v[3]);
```

Resposta:
Primeiro é impresso 0, porque os elementos de um array de int em Java são inicializados com zero. Em seguida, o acesso v[3] provoca uma ArrayIndexOutOfBoundsException, pois um array de 3 posições possui índices válidos de 0 a 2.
Detecção: o problema é detectado em execução.

Questão 5 — Rust

Código:

```
let s = String::from("oi");
let t = s;
println!("{} {}", s, t);
```

Resposta:
O programa não compila. Ao executar let t = s, a posse do valor é movida de s para t. O uso posterior de s é inválido e o compilador emite o erro E0382.
Detecção: o problema é detectado em compilação.

Questão 6 — C

Código:

```
union { int i; float f; } u;
u.f = 1.0f;
printf("%d\n", u.i);
```

Resposta:
A saída apresentada no material é 1065353216. A união livre permite gravar o valor como float e ler os mesmos bits como int. Os bits que representam 1.0f, quando interpretados como inteiro, resultam nesse valor.
Detecção: o problema nunca é detectado automaticamente.

Síntese

Os exemplos mostram diferentes filosofias das linguagens quanto à segurança de tipos e à detecção de erros. Rust detecta certos problemas ainda na compilação; Java detecta o acesso inválido ao array durante a execução; JavaScript, Go e C podem permitir situações problemáticas sem qualquer detecção automática. A principal conclusão é que escolhas de projeto dos tipos de dados afetam diretamente a segurança, a confiabilidade e o comportamento dos programas.